

© The Author(s), under exclusive license to APress Media, LLC, part of Springer Nature 2022

V. Sarcar, *Java Design Patterns*

https://doi.org/10.1007/978-1-4842-7971-7_2

2. Simple Factory Pattern

Vaskaran Sarcar¹

(1) Garia, Kolkata, India

This chapter covers the Simple Factory pattern.

Definition

It creates an object without exposing the instantiation logic to the client.

Concept

In object-oriented programming, a factory is an object that can create other objects. A factory can be invoked in many different ways, but most often it uses a method that can return objects with varying prototypes. Any subroutine that can help you create these new objects can be considered as a factory. Most importantly, it helps you abstract the process of object creation from the consumers of the application.

Real-Life Example

Consider a car manufacturing company that manufactures different models of a car. It must have a factory with different production units. Different models of cars are available so some parts are common across these models, such as company logo, name, etc. Some factory units produce common parts for all models. Other factory units produce model-specific parts. When they make the final product, they assemble the model-specific parts with the common parts. So, the goal of using a factory is to have a single production unit for similar parts. This ensures consistent product quality and functionality. As a result, when a part gets to the assembly line, it is certain that it will always behave the same way.

The Simple Factory pattern is the basis of the Factory Method and Abstract Factory patterns you'll learn in the next two chapters. So, you'll see a fine-tuned version of this example in the next chapter.

Let's consider a simpler example. When a kid demands a toy from their parents, the kid does not know how the parents will fulfill the demand. The parents in this case can be considered as a factory for this small kid. Think from the parents' point of view now. They can make the toy themselves or purchase the toy from a shop.

Computer World Example

This pattern is common to software applications, but before proceeding, note the following:

- The Simple Factory pattern is not treated as a standard design pattern in the GoF's famous book, but the approach is common to any application you write where you want to separate the code that varies a lot from the code that does not vary. It is assumed that you follow this approach in all applications you write.
- The Simple Factory pattern is considered the simplest form of the Factory Method pattern (and Abstract Factory pattern). So, you can assume that any application that follows either the Factory Method pattern or the Abstract Factory pattern also follows the concept of the Simple Factory pattern's design goals.
- You can consider the static method `getInstance()` of the `java.text.NumberFormat` class is an example of this category.

In the following implementation, I discuss this pattern with a common use case. Let's go through the implementation.

Implementation

These are the important characteristics of the following implementation:

- In this example, you are dealing with two different types of animals: dogs and tigers. You can assume that each type of animal can tell something about it, so you have two concrete classes called `Dog.java` and `Tiger.java`. Each implements the `Animal` interface. Here is a sample for your reference:

```
interface Animal{
    void displayBehavior();
}
class Dog implements Animal{
    public Dog() {
        System.out.println("\nA dog is created.");
    }
    public void displayBehavior(){
        System.out.println("It says: Bow-Wow.");
        System.out.println ("It prefers barking.");
    }
}
```

- The code for creating objects is in a different place (specifically, in a `factory` class). Using this approach, when you create either a dog or a tiger, you do not use the `new` operator in the client code (Q&A 2.2 discusses the reason for this in detail). So, in the client code, you see lines something like

```
Animal animal = factory.createAnimal("dog");
```

- In the same way, you can create a tiger.
- It means that to get an animal, the client invokes the `createAnimal()` method of the `AnimalFactory` object. Notice that you do not use the `new` operator in client code to create an object.

- You will further notice that the code that can vary is separated from the code that is least likely to vary. This approach can help you to remove tight coupling in the system. (How? Follow the Q&A Session.)

NoteTo make the example meaningful, I introduce method `displayBehavior()` in this demonstration. This method is not needed to demonstrate the Simple Factory pattern in this chapter or the Factory Method pattern in the next chapter. I use a console message in the constructor of the animal classes. These statements are sufficient to demonstrate these patterns. But as said, the use of the `displayBehavior()` method can make the example more meaningful. Invoking this method falls in the code region that does not vary in client code.

Class Diagram

Figure 2-1 shows the class diagram.

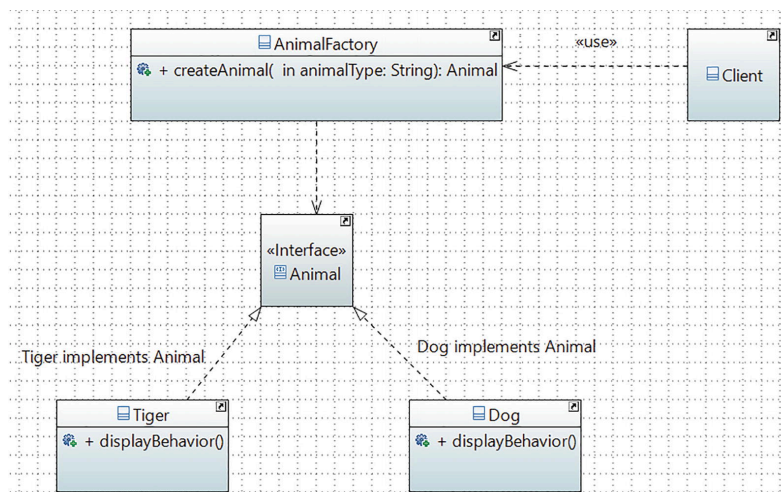


Figure 2-1 Class diagram

NoteI mention the usage and realization edges (or links) in this diagram. Next time onwards, I won't explicitly mention them because mentioning them repeatedly in the upcoming diagrams can make the overall structure clumsy.

Package Explorer View

Figure 2-2 shows the high-level structure of the program.

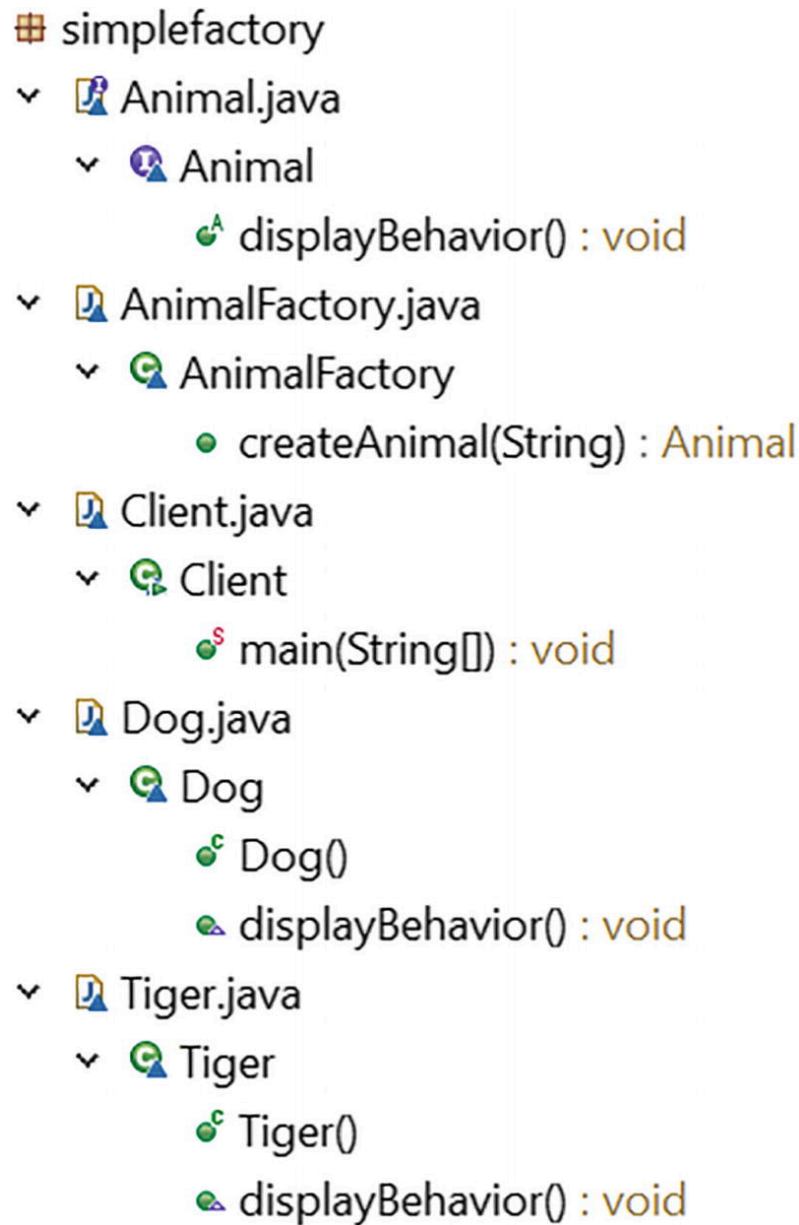


Figure 2-2 Package Explorer view

Demonstration

Here's the complete implementation. When you download the code, you will see that all parts of the program are separated and placed in the package `jdp3e.simplefactory`. Here's the implementation:

```
// Animal.java
interface Animal{
    void displayBehavior();
}

// Dog.java
class Dog implements Animal{
    public Dog(){
        System.out.println("\nA dog is created.");
    }
    public void displayBehavior(){
        System.out.println("It says: Bow-Wow.");
        System.out.println("It prefers barking.");
    }
}

// Tiger.java
class Tiger implements Animal{
```

```

public Tiger()
{
    System.out.println("\nA tiger is created.");
}

public void displayBehavior(){
    System.out.println("Tiger says: Halum.");
    System.out.println("It loves to roam in a jungle.");
}
}

// AnimalFactory.java
package jdp3e.simplefactory;
class AnimalFactory {
    public Animal createAnimal(String animalType){
        Animal animal;
        if (animalType.equals("dog")){
            animal = new Dog();
        }
        else if (animalType.equals("tiger")){
            animal = new Tiger();
        }
        else{
            System.out.println("You can create either a 'dog' or a 'tiger'. ");
            throw new IllegalArgumentException("Unknown animal cannot be instantiated.");
        }
        return animal;
    }
}

// Client.java
class Client {
    public static void main(String[] args) {
        System.out.println("*** Simple Factory
                                Demonstration***");

        AnimalFactory factory = new AnimalFactory();
        Animal animal = factory.createAnimal("dog");
        animal.displayBehavior();
        animal = factory.createAnimal("tiger");
        animal.displayBehavior();
    }
}

```

Output

You will see the following output when you run the program:

```

*** Simple Factory Demonstration***
A dog is created.
It says: Bow-Wow.
It prefers barking.
A tiger is created.
Tiger says: Halum.
It loves to roam in a jungle.

```

Analysis

Inside the `createAnimal()` method, you can pass either 'tiger' or 'dog'. Otherwise, you'll see a runtime exception. For example, if you append the following lines in the client code

```

animal = factory.createAnimal("cat"); //Exception

```

```
animal.displayBehavior();
```

you'll see an error like the following:

```
You can create either a 'dog' or a 'tiger'.
Exception in thread "main" java.lang.IllegalArgumentException: Unknown animal cannot
    at jdp3e.simplefactory.AnimalFactory.createAnimal(AnimalFactory.java:12)
    at jdp3e.simplefactory.Client.main(Client.java:16)
```

Q&A Session

2.1 The analysis section shows that you are passing String parameters and you do not ensure the type safety in this application. Is this correct?

Good catch. You are correct. But remember that in each chapter, the key focus is on a particular design pattern, not on these corner cases. You can always employ your preferred mechanism to ensure the type safety. For example, it is ok to use enums to guard this situation. Here is an alternative version of `AnimalFactory` that shows usage of an enum with the important changes in bold:

```
class AnimalFactory2 {
    public enum Type {DOG, TIGER};
    public Animal createAnimal(Type animalType){
        Animal animal;
        if (animalType.equals(Type.DOG))
        {
            animal = new Dog();
        }
        else if (animalType.equals(Type.TIGER))
        {
            animal = new Tiger();
        }
        // Remaining code is skipped
    }
}
```

To accommodate this change, here is an alternative version of the client code:

```
class Client2 {
    public static void main(String[] args) {
        System.out.println("*** Simple Factory Demonstration-2.***");
        AnimalFactory2 factory = new AnimalFactory2();
        Animal animal =
            factory.createAnimal(AnimalFactory2.Type.DOG);
        animal.displayBehavior();
        animal =
            factory.createAnimal(AnimalFactory2.Type.TIGER);
        animal.displayBehavior();
        // Compile-time error now
        // animal = factory.createAnimal("cat");
        // animal.displayBehavior();
    }
}
```

As a result, this time if you pass an incompatible animal such as 'cat' inside the `createAnimal()` method, you receive a compile-time error in-

stead of a runtime error. We all know that compile-time errors are always better compared to last-minute surprises such as runtime errors.

2.2 In this example, I see that the clients are delegating the object's creation through the Simple Factory pattern. But instead of this, they could directly create the objects with the `new` operator. Is this correct?

No. In this context, you need to remember the key reasons for using the Simple Factory pattern:

- One of the key object-oriented design principles is to separate the parts of your code that are most likely to change from the rest.
- In this case, only the creation process for objects changes. You can assume that these animals must tell something about themselves, and that part of the code does not need to vary inside the client code. So, in the future, if there is any change required in the creation process, you need to change only the `createAnimal()` method of the `AnimalFactory` class. The client code will not be affected because of those changes.
- You do not want to put lots of `if-else` blocks (or `switch` statements) inside the client code. This makes your client code clumsy.
- How you are creating the objects is hidden inside the client code. This kind of abstraction promotes security.

In short, sometimes the life-cycle management of the created objects must be centralized to ensure consistent behavior within the application. This cannot be done if the client is free to create a concrete object the way they wish.

2.3 What are the challenges associated with this pattern?

If you want to add a new animal or delete an existing animal, you need to modify the `createAnimal()` method. This process violates the Open/Closed Principle (which basically says that a code module should be open for extension but closed for modification) of the SOLID principles. (Refer to Chapter 1 if you need to recall the concept.)

2.4 Can you make the factory class static?

It depends on the programming language you use. In Java, you are not allowed to tag the word `static` with a top-level class. In other words, by design, the compiler will always complain about the top-level static classes in Java.

For example, in C#, you can have a static class, but you have to remember the restrictions associated with a static class. For example, you cannot inherit them, and so on. It can make sense when you deal with value objects that do not have an implementation class or a separate interface. It is also useful when you work with immutable classes and your factory class doesn't need to return a brand-new object each time you use it.

In short, a value object is an object whose equality is based on the values rather than the identity. The most important characteristic of a value ob-

ject is that it is immutable without an identity.

A simple real-life example can be given using five-rupee currency notes and five-rupee coins in India. Their money values are the same, but they are different instances.

In general, I do NOT vote for static factory class because they are basically promoters for global states, which are not ideal for object-oriented programming.

2.5 You could make the `createAnimal()` method static. Is this correct?

Yes. In fact, this is a common technique and is often termed as a static factory. The only advantage of this approach is that you can avoid instantiating objects from a class to use a method. But if you need to consider subclassing, you face troubles. In Chapter 3, you'll learn about the Factory Method pattern, and you'll see that you need to improve this program where a subclass needs to change the behavior of this method.

Final Comment

I told you earlier that you can consider the static method `getInstance()` of `java.text.NumberFormat` class as an example of this category. Now you understand the Simple Factory pattern. You can verify this easily in the Eclipse IDE. For example, right-click the `NumberFormat` class and then click Open Declaration. Scroll down to find the definition of the overloaded version of `getInstance(...)` method (see Figure 2-3).

```
private static NumberFormat getInstance(LocaleProviderAdapter adapter,
                                       Locale locale, Style formatStyle,
                                       int choice) {
    NumberFormatProvider provider = adapter.getNumberFormatProvider();
    return switch (choice) {
        case NUMBERSTYLE -> provider.getNumberInstance(locale);
        case PERCENTSTYLE -> provider.getPercentInstance(locale);
        case CURRENCYSTYLE -> provider.getCurrencyInstance(locale);
        case INTEGERSTYLE -> provider.getIntegerInstance(locale);
        case COMPACTSTYLE -> provider.getCompactNumberInstance(locale, formatStyle);
        default -> null;
    };
}
```

Figure 2-3 An overloaded version of the `getInstance(...)` method inside the `NumberFormat` class

Don't be surprised! It is a new kind of `case` level in switch statements where you can use "arrow case" labels instead of "colon case" labels (it is available in JDK13 onwards). In the same way, you can verify other examples in this book.

Previous chapter

< [1. Understanding SOLID Principles](#)

Next chapter

[Part II. The Gang of Four \(GoF\) Design Patterns](#) >